

Agentic GraphRAG for accounting regulatory *research*.

A weekend on retrieval, graphs, and agents — over real European regulation.

HOSTED BY
Aalto University



CHALLENGE BY
Taxxa AI Oy

github.com/taxxa-ai/aalto-hackaton-2026



WHO WE ARE

The Second Brain for modern accounting.

Trusted by 4,300+ professionals across 7 EU countries. We ingest official regulation, keep it in sync, and expose it through an LLM-native interface.

The hard part — the part we want to talk about today — is making the answers actually correct.



TODAY: RAG

Knowledge research, today: a standard RAG pipeline.

SCRAPE

01

Source ingestion

400+ official
sources: legislation,
tax authority
bulletins, court
rulings.

CHUNK

02

Segmentation

Split each
document into
passages a
language model can
attend to in a single
context window.

EMBED

03

Vectorization

Pass each chunk
through an
embedding model;
store the resulting
vector in an ANN
index.

RETRIEVE

04

Top-k search

At query time,
embed the question
and fetch the
nearest k passages.
Optionally rerank.

GENERATE

05

Grounded answer

Stuff retrieved
passages into the
prompt, ask the
LLM to answer with
citations back to
source.



Where naive RAG breaks on regulatory text.

CHUNKING

a.

Semantic units don't respect token boundaries

Fixed-size or paragraph-split chunkers cut clauses mid-sentence. Exceptions and definitions land in different chunks than the rule they qualify.

RANKING

b.

Cosine similarity is jurisdiction-blind

A repealed 2014 article and a binding 2024 decree can sit at the same distance from the query. Recency, authority, scope — all flattened into one score.

STRUCTURE

c.

Inter-document references aren't followed

"See §102 AVL" is just text to an ANN index. The retriever can't hop along citation edges to pull the actual referenced clause.

COMPOSITION

d.

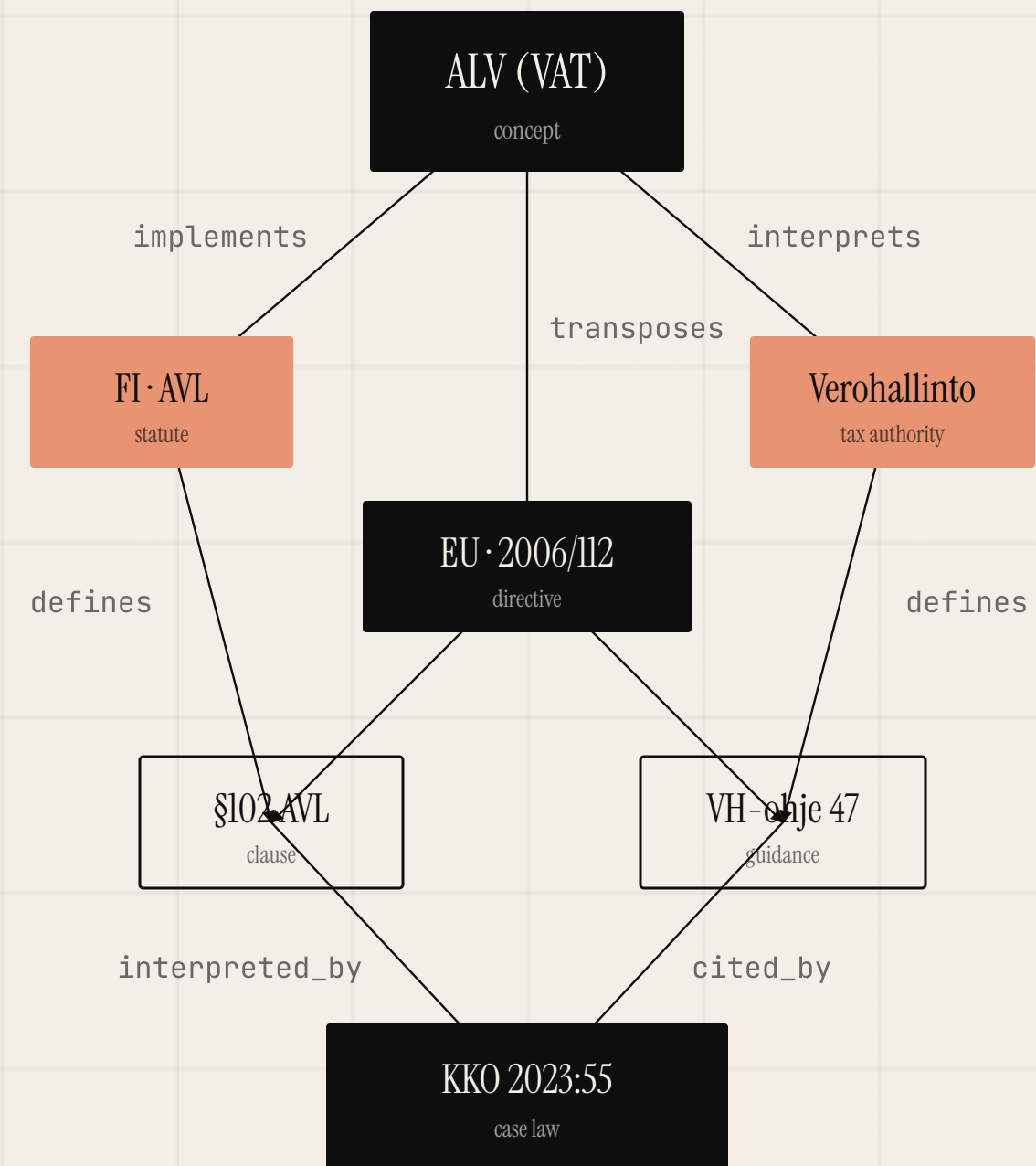
Multi-hop questions get one-hop answers

"Is X deductible for an exempt entity?" needs a rule, an exception, and a definition — from three documents. Top-k just returns three near-duplicates of the question.



Give retrieval a *structure* to walk.

Entities as nodes. References, hierarchies, repeals, jurisdiction as typed edges. Retrieval becomes a constrained graph traversal instead of a cosine race over a flat index.



FROM RAG TO GRAPH RAG

Building the graph — four open problems.

NER

01

Entity extraction

Pull domain entities out of regulation text: articles, taxes, agencies, deadlines, monetary thresholds. Domain-specific NER beats generic models — you have the corpus.

RELATIONS

02

Relation extraction

Type the edges. Amends · repeals · transposes · references · applies-when. Mix rule-based parsers (clear citation forms) with LLM-as-extractor for the looser cases.

HIERARCHY

03

Section-aware parsing — not chunking

Drop fixed-size windows. Use the regulation's own units: book → title → article → paragraph. The hierarchy carries scope and inheritance; let retrieval walk it.

RETRIEVAL

04

Hybrid retrieval — embeddings + edges

Embed each node's text — its definition, summary or surface forms. A query finds the right entry points by vector similarity; the graph carries the walk from there. Fuzzy entry, structured traversal.



FROM GRAPHRAG TO AGENTS

Where agents earn their keep.

Hard regulatory questions need decomposition, lookups, and second-guessing — the kind of work an orchestrator of small specialised agents can handle.

PLANNER

01

Query planning subagent

Decompose "is X deductible?" into: identify entity → locate governing statute → check exceptions → check case-law overrides. Picks a traversal strategy over the graph.

EXTRACTOR

02

LLM-as-extractor on demand

When rule-based parsers stall on a fuzzy citation or implicit reference, hand the passage to a small LLM agent that returns a typed relation — with a confidence score and a justification.

VERIFIER

03

Claim-to-source critic

Walks each claim back to a node in the graph. Surfaces conflicting paths (e.g. statute vs. later case law), refuses to commit when sources disagree.

CLARIFIER

04

Disambiguation subagent

Most regulatory questions are underspecified — missing jurisdiction, tax year, entity type. Detects the gaps and either asks back, or runs the query under explicit assumptions surfaced in the answer.



Questions?



We're around *all weekend.*

Happy to pair on code, look at your repo, or argue about graph schemas. Hop on Slack or send a note.

REPO

github.com/taxxa-ai/aalto-hackaton-2026

MAIL

stephane@taxxa.ai

SLACK · JOIN THE CHANNEL

Scan to join

